

Table 1: DEQ achieves strong performance on the long-range copy-memory task.

	Models (Size)			
	DEQ-Transformer (ours) (14K)	TCN [7] (16K)	LSTM [26] (14K)	GRU [14] (14K)
Copy Memory $T=400$ Loss	3.5e-6	2.7e-5	0.0501	0.0491

Table 2: DEQ achieves competitive performance on word-level Penn Treebank language modeling (on par with SOTA results, without fine-tuning steps [34]). [†]The memory footprints are benchmarked (for fairness) on input sequence length 150 and batch size 15, which does not reflect the actual hyperparameters used; the values also do *not* include the memory for word embeddings.

Word-level Language Modeling w/ Penn Treebank (PTB)				
Model	# Params	Non-embedding model size	Test perplexity	Memory [†]
Variational LSTM [22]	66M	-	73.4	-
NAS Cell [55]	54M	-	62.4	-
NAS (w/ black-box hyperparameter tuner) [32]	24M	20M	59.7	-
AWD-LSTM [34]	24M	20M	58.8	-
DARTS architecture search (second order) [29]	23M	20M	55.7	-
60-layer TrellisNet (w/ auxiliary loss, w/o MoS) [8]	24M	20M	57.0	8.5GB
DEQ-TrellisNet (ours)	24M	20M	57.1	1.2GB

conventional deep nets) for word-level language modeling, we show that DEQ achieves competitive (or better) performance even when compared to SOTA methods (of the same model size, both weight-tied and not) while using significantly less memory. We provide a more detailed introduction of the tasks and datasets in Appendix F.

Setting. Both instantiations of DEQ use Broyden’s method [10] to avoid direct computation of the inverse Jacobian, as described in Section 3.1.3. We note that the use of DEQ implicitly introduces a new “hyperparameter” – the stopping criterion for Broyden iterations. During training, we set this tolerance ε of forward and backward passes to $\varepsilon = \sqrt{T} \cdot 10^{-5}$ and $\sqrt{T} \cdot 10^{-8}$, respectively. At inference, we relax the tolerance to $\varepsilon = \sqrt{T} \cdot 10^{-2}$ (or we can use a smaller maximum iteration limit for Broyden’s method; see discussions later). For the DEQ-TrellisNet instantiation, we roughly follow the settings of [8]. For DEQ-Transformers, we employ the relative positional embedding [16], with sequences of length 150 at both training and inference on the WikiText-103 dataset. Implementations and pretrained models can be found at <https://github.com/locuslab/deq>.

5.1 Copy Memory Task

The goal of the *copy memory task* is simple: to explicitly test a sequence model’s ability to exactly memorize elements across a long period of time (see Appendix F). As shown in Table 1, DEQ demonstrates good memory retention over relatively long sequences ($T = 400$), with substantially better results than recurrent architectures such as LSTM/GRU (consistent with the findings in [7]).

5.2 Large-Scale Language Modeling

One issue encountered in prior works that take a continuous view of deep networks [11, 24] is the challenge of scaling these approaches to real, high-dimensional, large-scale datasets. In this subsection, we evaluate the DEQ approach on some large-scale language datasets and investigate its effectiveness as a practical “implicit-depth” sequence model.

Performance on Penn Treebank. Following the set of hyperparameters used by [8] for TrellisNet, we evaluate the DEQ-TrellisNet instantiation on word-level language modeling with the PTB corpus. Note that without an explicit notion of “layer”, we do not add auxiliary losses, as was done in [8]. As shown in Table 2, when trained from scratch, the DEQ-TrellisNet achieves a test perplexity on par with the original deeply supervised TrellisNet.

Performance on WikiText-103. On the much larger scale WT103 corpus (about 100x larger than PTB), the DEQ-TrellisNet achieves better test perplexity than the original deep TrellisNet. For the Transformer instantiation, we follow the design of the Transformer-XL model [16]. We specifically compare to a “medium” Transformer-XL model (the largest released model that can fit on GPUs)

Table 3: DEQ-based models are competitive with SOTA deep networks of the same model size on the WikiText-103 corpus, with significantly less memory. [†]See Table 2 for more details on the memory benchmarking. Transformer-XL models are not weight-tied, unless specified otherwise.

Word-level Language Modeling w/ WikiText-103 (WT103)				
Model	# Params	Non-Embedding Model Size	Test perplexity	Memory [†]
Generic TCN [7]	150M	34M	45.2	-
Gated Linear ConvNet [17]	230M	-	37.2	-
AWD-QRNN [33]	159M	51M	33.0	7.1GB
Relational Memory Core [40]	195M	60M	31.6	-
Transformer-XL (X-large, adaptive embed., on TPU) [16]	257M	224M	18.7	12.0GB
70-layer TrellisNet (+ auxiliary loss, etc.) [8]	180M	45M	29.2	24.7GB
70-layer TrellisNet with <i>gradient checkpointing</i>	180M	45M	29.2	5.2GB
DEQ-TrellisNet (ours)	180M	45M	29.0	3.3GB
Transformer-XL (medium, 16 layers)	165M	44M	24.3	8.5GB
DEQ-Transformer (medium, ours).	172M	43M	24.2	2.7GB
Transformer-XL (medium, 18 layers, adaptive embed.)	110M	72M	23.6	9.0GB
DEQ-Transformer (medium, adaptive embed., ours)	110M	70M	23.2	3.7GB
Transformer-XL (small, 4 layers)	139M	4.9M	35.8	4.8GB
Transformer-XL (small, weight-tied 16 layers)	138M	4.5M	34.9	6.8GB
DEQ-Transformer (small, ours).	138M	4.5M	32.4	1.1GB

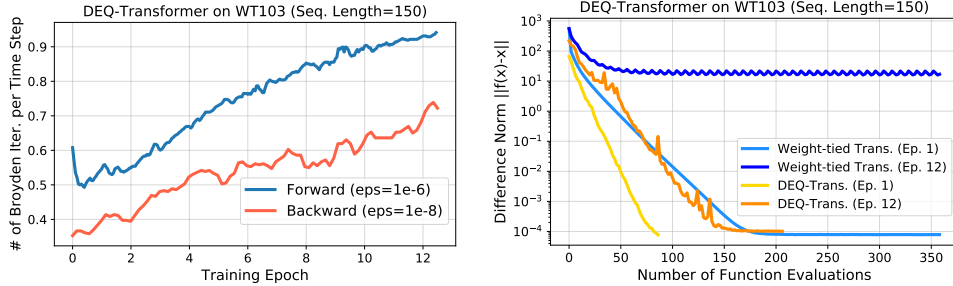


Figure 2: Left: number of Broyden iterations in forward and backward passes gradually grows with epochs. Right: DEQ-Transformer finds the equilibrium in a stable and efficient manner (whereas the deep transformer could oscillate around the fixed point, even when one exists).

and a “small” Transformer-XL model, while noting that the largest Transformer-XL network has massive memory requirements (due in part to very wide hidden features, batch sizes, and training-time sequence lengths, which would not be decreased by a DEQ) and can only be trained on TPUs [16]. In Table 3, we show that the DEQs yield competitive performance, outperforming prior SOTA approaches such as [16] on similar model sizes while consuming much less memory during training.

Memory footprint of DEQ. For conventional deep networks with L layers, the training memory complexity is $O(L)$ since all intermediate activations are stored for backpropagation. In comparison, DEQs have an $O(1)$ (i.e., constant) memory footprint due to the root-finding formulation. We benchmark the reduced memory consumption in the last column of Tables 2 and 3, with controlled sequence lengths and batch sizes for fairness. On both instantiations, the DEQ approach leads to an over 80% (up to 88%) reduction in memory consumption by the model (excluding word embeddings, which are orthogonal to the comparison here). Moreover, we empirically verify (using a 70-layer TrellisNet) that DEQ consumes even less memory than gradient checkpointing [12], a popular technique that reduces the memory required to train a layer-based model to $O(\sqrt{L})$. Note that the DEQ’s memory footprint remains competitive even when compared with baselines that are not weight-tied (a reduction of over 60%), with similar or better accuracy.

Initialization of DEQ. To train DEQ models, it is critical to ensure that the model is stable, such that the equilibrium state can be reliably approximated via quasi-Newton methods. While we found that the most commonly used initialization schemes with small values (around 0) suffice, it is generally important to make sure that DEQ starts with a small operator norm in the weight matrices. For both DEQ-TrellisNet and DEQ-Transformer, we observe that they are not sensitive to any specific initialization scheme since non-linearities such as $\sigma/\tanh$ and LayerNorm also help make f_θ contractive (and stable). We initialize the parameters of f_θ by sampling from $\mathcal{N}(0, 0.05)$.

Table 4: Runtime ratios between DEQs and corresponding deep networks at training and inference ($> 1\times$ implies DEQ is slower). The ratios are benchmarked on WikiText-103.

DEQ / 18-layer Transformer		DEQ / 70-layer TrellisNet	
Training	Inference	Training	Inference
$2.82\times$	$1.76\times$	$2.40\times$	$1.64\times$

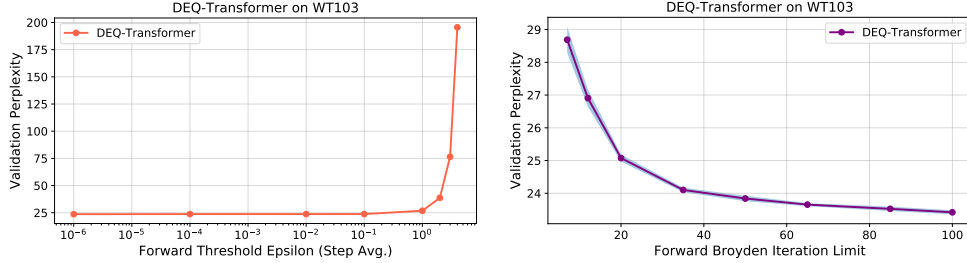


Figure 3: DEQ can be accelerated by leveraging higher tolerance ε (left) or a lower Broyden iteration limit (right). In general, poor estimates of the equilibrium can hurt DEQ performances.

Convergence to equilibrium. The deep equilibrium model does not have “layers”. One factor that affects computation time in DEQs is the number of Broyden iterations in forward/backward passes, where each forward Broyden step evaluates f_θ once, and a backward step computes a vector-Jacobian product. We find that in general the number of Broyden iterations gradually increases with training epochs (Figure 2, left, where the y -axis is computed by $\frac{\text{Total Broyden Iterations}}{\text{Sequence Length}}$), an observation similar to the one reported for training Neural ODEs [11]. One factor contributing to this phenomenon could be that the training pushes the operator norm of J_{f_θ} to larger values, making the fixed point harder to solve. Meanwhile, the backward pass requires much fewer iterations than the forward, primarily due to the simplicity of the linear system in Eq. (11). We also find that DEQs can almost always converge to the sequence-level fixed point, much more efficiently than original weight-tied transformers (Figure 2, right). Note that after 12 epochs, deeply stacked self-attention tends to oscillate around the fixed point, while DEQs exhibit stable convergence with the quasi-Newton method.

Broyden iterations and the runtime of DEQ. Unlike conventional deep networks that come with a fixed number L of layers, the runtime of DEQ depends strongly on the number of Broyden steps to reach the equilibrium. Therefore, it’s challenging to fairly compare the runtimes of implicit-depth models like DEQ with those of corresponding weight-tied deep networks (e.g., using higher depth necessarily takes longer to run). Ideally, the values of ε should be as small as possible so as to ensure that the analytical gradients from Theorem 1 are accurate. However, we empirically observe that using a higher ε or a lower iteration limit allows the DEQ to be trained and evaluated much faster with only a small degradation in performance. For instance, generally we find $\varepsilon < 0.1$ or an iteration limit of 30 (on sequence length 75) to be sufficient for competitive performance. Figure 3 visualizes this tradeoff on a medium DEQ-Transformer (without adaptive embedding). Note that accuracy quickly diverges when tolerance ε is too large (Figure 3, left), suggesting that a poor estimate of the equilibrium can hurt DEQ performances. Table 4 provides approximate runtimes for competitive-accuracy DEQs on WikiText-103. DEQs are typically slower than layer-based deep networks.

Additional empirical remarks as well as training tips are provided in Appendix E.

6 Conclusion

Deep networks have predominantly taken the form of stacks of layers. We propose the deep equilibrium approach (DEQ), which models temporal data by directly solving for the sequence-level fixed point and optimizing this equilibrium for better representations. DEQ needs only $O(1)$ memory at training time, is agnostic to the choice of the root solver in the forward pass, and is sufficiently versatile to subsume drastically different architectural choices. Our experiments have shown that DEQs have good temporal memory retention, are able to scale to realistic, large-scale sequence tasks, and perform competitively with, or slightly outperform, SOTA methods. Overall, we believe that the DEQ approach provides an interesting and practical new perspective on designing and optimizing sequence models.